



Structured Programming Concepts

(26CSE0105)



Flowcharts

Prepared By: Dr. Punit Soni

Department of Computer Science and Engineering

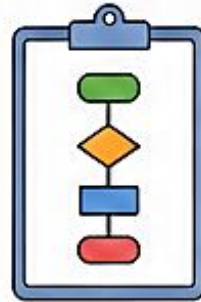
Chitkara University Institute of Engineering and Technology

Chitkara University, Punjab

Topics to be covered

- ***Flowcharts***
- ***Standard Flowchart Symbols***
- ***Flowchart Designing implementation***
- ***Pseudocode***
- ***Difference between Algorithm, Flowchart and Pseudocode***
- ***Structured vs Unstructured Programming***
- ***Computational Problem Solving.***

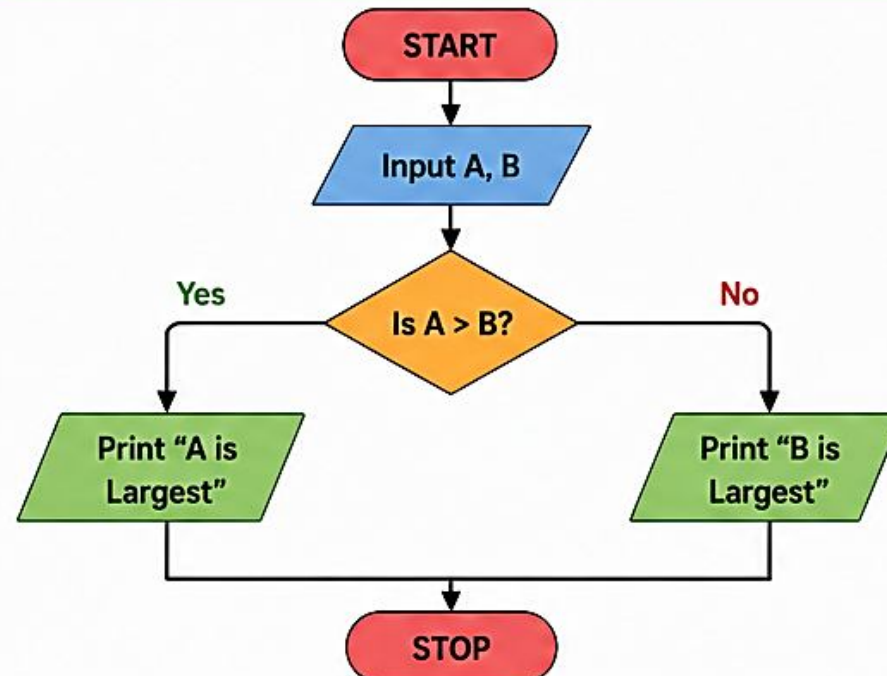
Flowcharts



WHAT IS A FLOWCHART?

A flowchart is a graphical representation of an algorithm or process. It uses standard symbols connected by arrows to show the sequence of steps to solve a problem.

EXAMPLE: FLOWCHART TO FIND THE LARGEST OF TWO NUMBERS





ADVANTAGES OF FLOWCHARTS

- Easy to understand and communicate.
- Helps in analyzing the problem.
- Acts as a blueprint for coding.
- Helps in debugging and testing.
- Improves documentation.



Flowcharts Symbols



SYMBOL	NAME	PURPOSE	EXAMPLE
	Terminator (Start/Stop)	Indicates the beginning or end of the flowchart.	
	Input / Output	Represents data input or output operations.	
	Process	Represents a processing step or instruction.	
	Decision	Represents a decision or condition.	
	Flow Line (Arrow)	Shows the direction of flow from one step to another.	
	On-Page Connector	Connects flowlines on the same page.	
	Off-Page Connector	Connects flowlines on different pages.	
	Annotation / Comment	Used to add explanatory notes or comments.	

Flowchart Designing

- **Flowchart design** translates **logical problem-solving steps** into a **visual blueprint** before mapping directly to C programming language syntax.
- **Designing a flowchart** establishes control flows to **easily structuralize clean code** like;

1. SEQUENTIAL STRUCTURE

Flowchart View: Rectangles and parallelograms stacked vertically with straight arrows pointing down.

2. DECISION MAKING (BRANCHING)

Flowchart View: A diamond box with one entry path and two separate outgoing path arrows ("Yes" and "No").

3. REPETITION STRUCTURE (LOOPING)

Flowchart View: An arrow loops back from a lower process box up to an earlier decision diamond, forming a physical loop.

Flowchart Implementation



1. Sequential Structure

Statement: Add two numbers.

Algorithm:

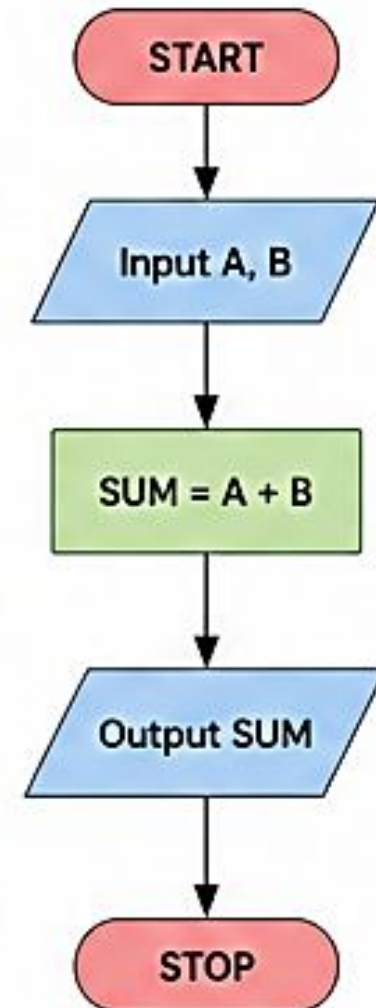
- **START**
- **Take two numbers (say: A, B) as input.**
- **Add them and store result in SUM.**
- **Display/Print SUM.**
- **STOP**



Key Point:

Instructions are executed one after another in the same order.

FLOWCHART VIEW



Flowchart Implementation



2. Decision Making (Branching)

Statement: Check Whether person is eligible for voting or not.

Algorithm:

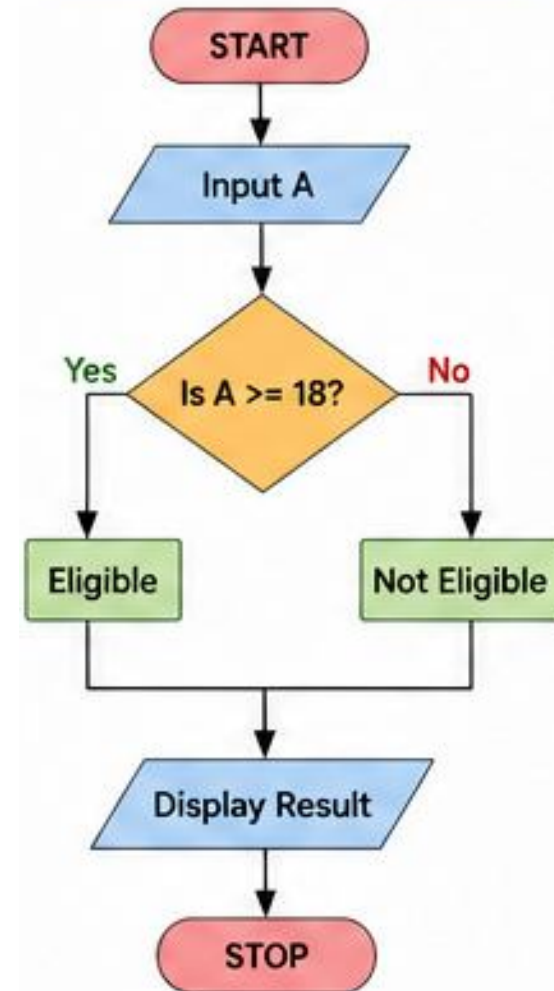
- **START**
- **Take input as age (say: A).**
- **Check If $A \geq 18$ then eligible else not eligible.**
- **Display/Print result.**
- **STOP**



Key Point:

The program chooses a path based on a condition.

FLOWCHART VIEW



Flowchart Implementation



3. Repetition Structure (Looping)

Statement: Find the sum of 5 numbers.

Algorithm:

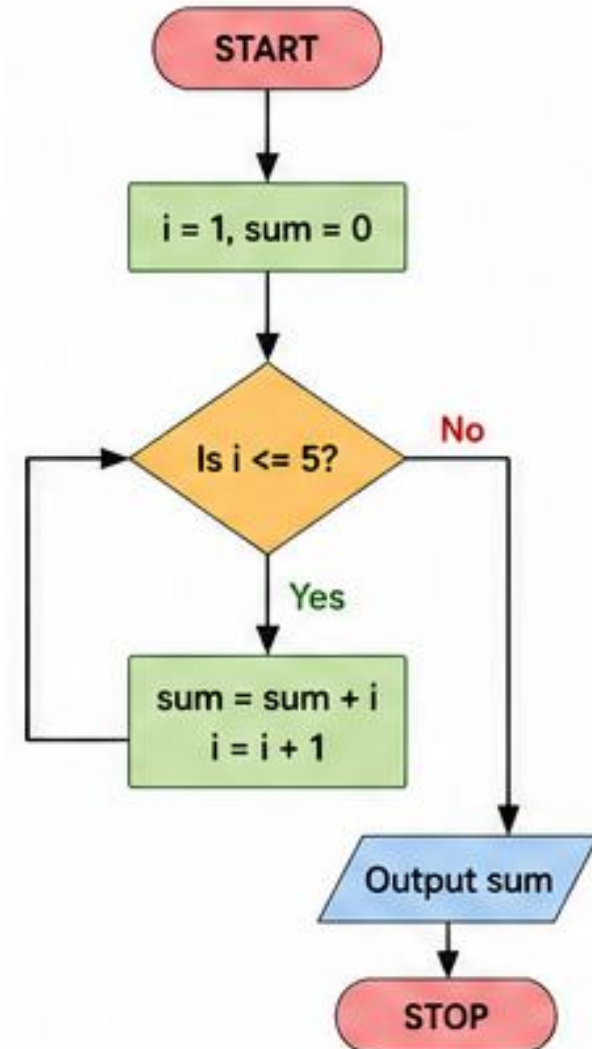
- **START**
- 1. Initialize **$i = 1$** and **$SUM = 0$** .
- 2. Check If **$i \leq 5$** then
 - **ADD SUM and i and store into SUM.**
 - **Increment i by 1 and goto step 2.**
- 3. Else Display/Print **SUM** .
- **STOP**



Key Point:

The program repeats a block of code multiple times.

FLOWCHART VIEW



Flowchart Designing (Exercise)

- *Check the entered character is Vowel or Consonant.*
- *Check the number is Even or Odd.*
- *Find the largest among two different numbers entered by the user.*
- *Find the largest among three different numbers entered by the user.*
- *Find the Factorial of a Number.*

Pseudo Code

WHAT IS PSEUDOCODE?

- Pseudocode is an informal way to write the steps of an algorithm.
- It is independent of any programming language syntax.
- It helps in planning and designing the logic before coding.



CHARACTERISTICS

- ✓ Easy to read and understand.
- ✓ Language independent.
- ✓ Focuses on logic, not on syntax.
- ✓ Helps in debugging and communication.
- ✓ Useful for both beginners and professionals.
- ✓ Acts as a bridge between human thinking and programming.



Pseudo Code (contd..)

GENERAL GUIDELINES FOR WRITING PSEUDOCODE

- Use meaningful names for variables and constants.
- Write statements in logical order.
- Indent to show structure (loops, decisions, etc.).
- Use simple English words.
- One statement per line.
- Use comments (::) to explain the logic if needed.



Pseudo Code (contd..)

COMMON PSEUDOCODE KEYWORDS	
Keyword	Meaning / Use
START / END	Marks the beginning and end of the algorithm.
INPUT / READ	Reads input values.
OUTPUT / PRINT	Displays output.
DECLARE	Declares variables.
IF – THEN – ELSE	Decision making.
WHILE / REPEAT UNTIL	Repetition (looping).
FOR	Count-controlled loop.
AND, OR, NOT	Logical operators.
← (Assignment)	Assigns value to a variable.
:: (Comment)	For remarks or explanation.

Pseudo Code (contd..)

PSEUDOCODE EXAMPLE (SUM OF N NUMBERS)

```
1.  START
2.  DECLARE n, i, sum
3.  INPUT n
4.  sum ← 0, i ← 1
5.  WHILE i ≤ n DO
6.      sum ← sum + i
7.      i ← i + 1
8.  ENDWHILE
9.  OUTPUT "Sum = ", sum
10. END
```

Pseudo Code (contd..)



EXAMPLE: PSEUDOCODE TO FIND THE LARGEST OF TWO NUMBERS

```
1.  START
2.  DECLARE num1, num2, largest
3.  INPUT num1, num2
4.  IF num1 > num2 THEN
5.      largest ← num1
6.  ELSE
7.      largest ← num2
8.  ENDIF
9.  OUTPUT "The largest number is: ", largest
10. END
```

Pseudo Code



ADVANTAGES

- ✓ Improves problem-solving skills.
- ✓ Acts as a blueprint for coding.
- ✓ Makes the program easy to understand.
- ✓ Helps in identifying errors early.
- ✓ Facilitates communication among developers.



APPLICATIONS

- Planning and designing algorithms.
- Documenting program logic.
- Teaching and learning programming.
- Translating logic into any programming language.



Pseudo Code (Exercise)

- *Check the entered character is Vowel or Consonant.*
- *Check the number is Even or Odd.*
- *Check the number is Positive, Negative, or Zero.*
- *Find the largest among three different numbers entered by the user.*
- *Find the Factorial of a Number.*

Comparison Chart



BASIS OF COMPARISON	 ALGORITHM	 FLOWCHART	 PSEUDOCODE
 Definition	A step-by-step, finite sequence of instructions to solve a problem.	A graphical representation of an algorithm using standard symbols and arrows.	A high-level description of an algorithm using plain English-like statements.
 Representation	Written in natural language (step-by-step text).	Graphical (boxes, arrows, and symbols).	Textual (structured English-like statements).
 Readability	Easy to read for simple problems, can be lengthy for complex problems.	Easy to understand visually, even for complex problems.	Easy to read and understand, closer to programming logic.
 Use of Symbols	No symbols used.	Uses standard symbols (terminator, process, decision, etc.).	No graphical symbols, uses keywords and indentation.
 Language Dependence	Language independent.	Language independent.	Language independent.
 Purpose / Use	To describe the logic in a simple, step-by-step manner.	To visualize the flow of control and process clearly.	To bridge the gap between natural language and programming code.
 Modification	Easy to modify.	Time-consuming to modify for large/complex problems.	Relatively easy to modify.
 Best Suited For	Small to medium problems.	Understanding and communicating complex workflows.	Planning before coding (any size of problem).

Example: Factorial of a number

Algorithm (Factorial of n)

- 1. Start**
- 2. Read number n**
- 3. Set $fact = 1$**
- 4. Repeat steps 5–6 while $i \leq n$**
- 5. $fact = fact * i$**
- 6. Increment i by 1**
- 7. Print $fact$**
- 8. Stop**

Pseudocode

BEGIN

INPUT n

$fact \leftarrow 1$

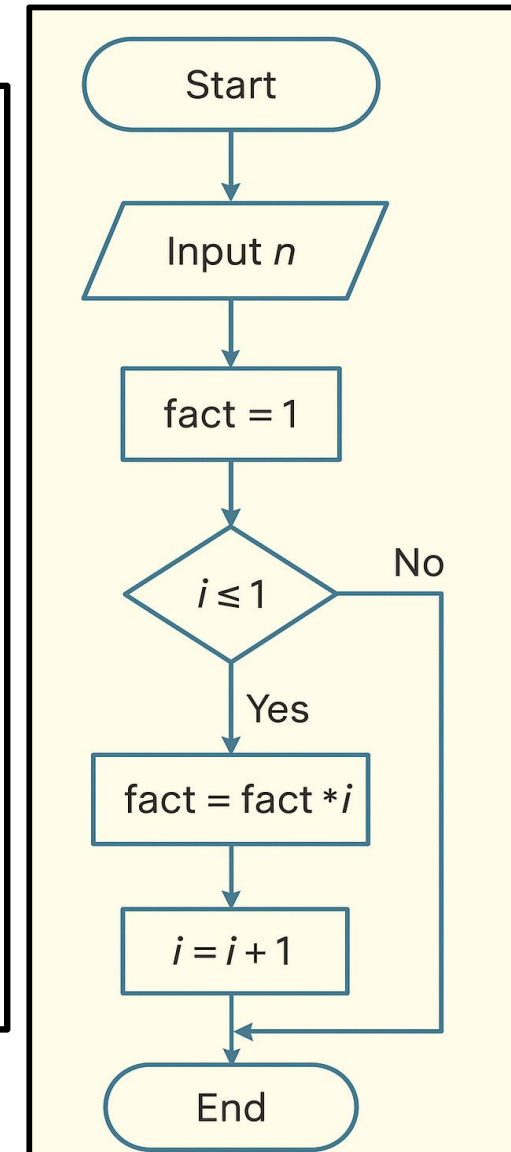
FOR $i \leftarrow 1$ TO n DO

$fact \leftarrow fact * i$

END FOR

OUTPUT $fact$

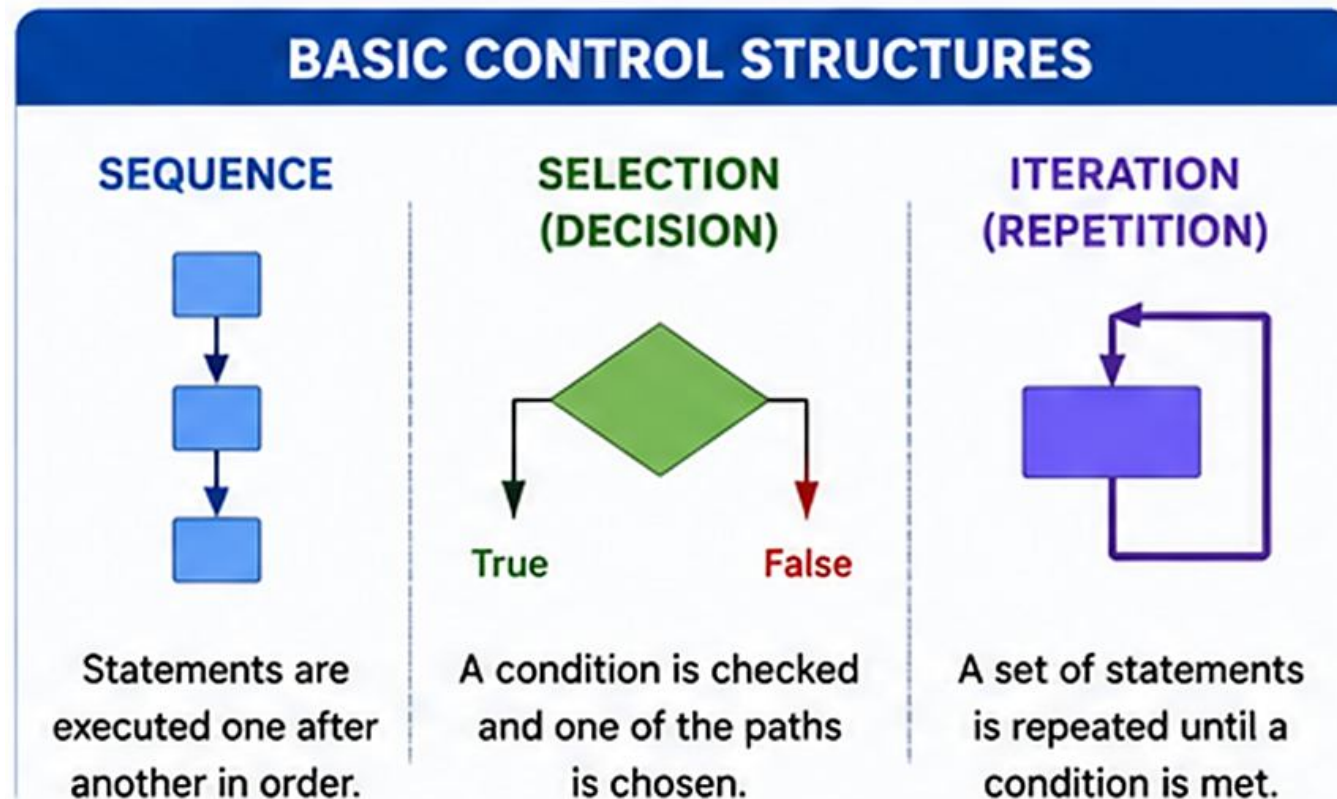
END



Structured Programming



- **Structured programming**, or **Modular programming**, is a programming paradigm that emphasizes **clear, logical structures** for the programs.
- It breaks the program into **smaller modules/ functions**.



Structured Programming Features



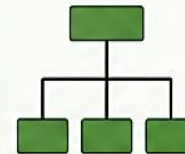
1 MODULARITY

- The program is divided into smaller modules or functions.
- Each module performs a specific task.
- Promotes code reusability and simplifies debugging.
- Makes the program easier to develop, test and maintain.



2 TOP-DOWN DESIGN

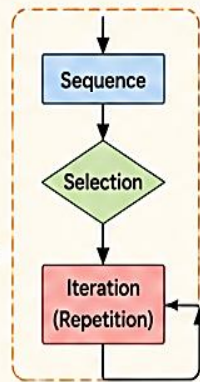
- The program is designed from a high level to a low level.
- The main problem is divided into smaller sub-problems.
- Each sub-problem is further refined until a detailed solution is obtained.
- Makes problem solving systematic and organized.



3 CONTROL FLOW STRUCTURES

Structured programming uses only three basic control structures:

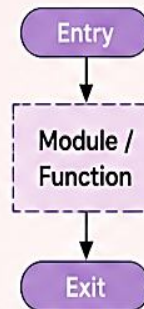
- **Sequence** – Executes statements one after another.
- **Selection (Decision)** – Chooses between alternative paths.
- **Iteration (Repetition)** – Repeats a set of statements.



STRUCTURED PROGRAMMING

4 SINGLE-ENTRY, SINGLE-EXIT

- Each module or function has one entry point and one exit point.
- Control enters the module at the top and exits at the bottom.
- Prevents complexity caused by multiple entry or exit points.
- Improves program correctness and reliability.



5 IMPROVED READABILITY AND MAINTAINABILITY



- The structured approach makes programs easy to read and understand.



- Easier to test and debug.
- Errors can be located and corrected quickly.



- Easier to modify and enhance.
- Reduces the cost and time of maintenance.

Structured Vs Unstructured Prog.

STRUCTURED PROGRAMMING	UNSTRUCTURED PROGRAMMING
A disciplined programming approach that uses sequence, selection and iteration to control the flow of execution.	A programming approach that does not follow a well-defined structure. It uses unrestricted jumps (GOTO) to control the flow of execution.
Follows a top-down approach.	Follows a bottom-up or ad-hoc approach.
Uses only three basic control structures: 1. Sequence 2. Selectnce (Decision) 3. Iteration (Repetition)	Uses GOTO statements for jumps. No fixed control structure.
Flow of control is clear and well-defined .	Flow of control is unclear and unpredictable .
Highly modular . Program is divided into functions or modules.	Not modular . Large program written as a single block of code.
High readability . Easy to read and understand.	Low readability . Difficult to read and understand.
Easy to test, debug and maintain.	Difficult to test, debug and maintain.
More reliable and less prone to errors.	Less reliable and more prone to errors.
Well suited for large and complex programs.	Not suitable for large and complex programs.
C, C++, Java, Python, Pascal, etc.	Early BASIC , some old assembly language programs.

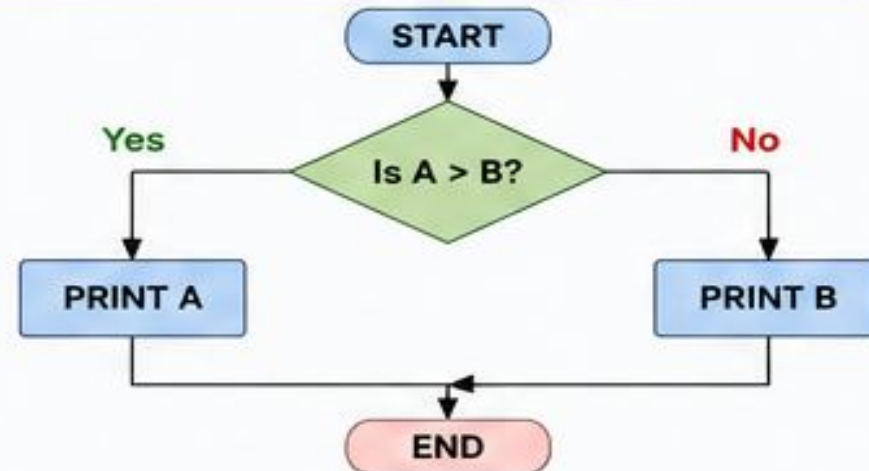
Structured Vs Unstructured Prog.



EXAMPLE: FIND THE LARGEST OF TWO NUMBERS

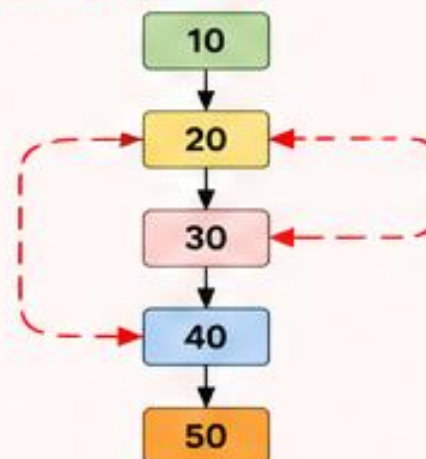
Structured Programming (Using IF-ELSE)

```
IF A > B THEN  
    PRINT A  
ELSE  
    PRINT B  
ENDIF
```



Unstructured Programming (Using GOTO)

```
10 INPUT A, B  
20 IF A > B THEN GOTO 40  
30 PRINT B  
   GOTO 50  
40 PRINT A  
50 END
```



GOTO makes the flow of control unstructured and difficult to follow.

Computational Problem Solving

- A **complete pipeline** from a **raw problem statement** to a **tested, working solution**.



- To solve a **computational problem**, a programmer must advance through **four distinct stages**:

- ❖ **Identify inputs**, required **constraints**, and expected **outputs**.
- ❖ **Create** a **step-by-step**, blueprint using **pseudo-code** or **flowcharts**.
- ❖ **Translate** the **blueprints** into **standard C syntax** using appropriate DS.
- ❖ **Compile, link** library dependencies, and **test** against edge cases to eliminate syntax and logic al bugs.

Computational Problem Solving

- **Problem:** Find the largest of three numbers.

1

Define Problem

Given three numbers entered by a user, determine and display the largest of the three.

2

Analyze

Inputs: a, b, c (numeric).

Output: the largest value.

Must handle the case where numbers are equal.

3

Design Algorithm

Compare a, b, c pairwise:

1. If $a > b$ and $a > c \rightarrow \text{largest} = a$
2. Else if $b > a$ & $b > c \rightarrow \text{largest} = b$
3. Else $\rightarrow \text{largest} = c$

4

Flowchart / Pseudocode

BEGIN

INPUT a, b, c

IF $a > b$ AND $a > c$ THEN

$\text{largest} \leftarrow a$

ELSE IF $b > a$ AND $b > c$ THEN

$\text{largest} \leftarrow b$

ELSE $\text{largest} \leftarrow c$

PRINT largest

END

- **Problem:** Find the largest of three numbers.

5 Write Code

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

if a > b and a > c:
    largest = a
elif b > a and b > c:
    largest = b
else:
    largest = c

print("The largest number is:", largest)
```

Computational Problem Solving

- **Problem:** Find the largest of three numbers.

6 Test & Debug

Input (a, b, c)	Expected	Result
5, 9, 3	9	Pass
10, 10, 8	10	Pass
-2, -5, -1	-1	Pass
7, 7, 7	7	Pass

7 Deploy Solution Once all test cases pass, the program is packaged and made available for end users — completing the pipeline.



Thank You!

